

中山大学网络空间安全学院必修课程

第3节 表达式

C Programming

岳 晟

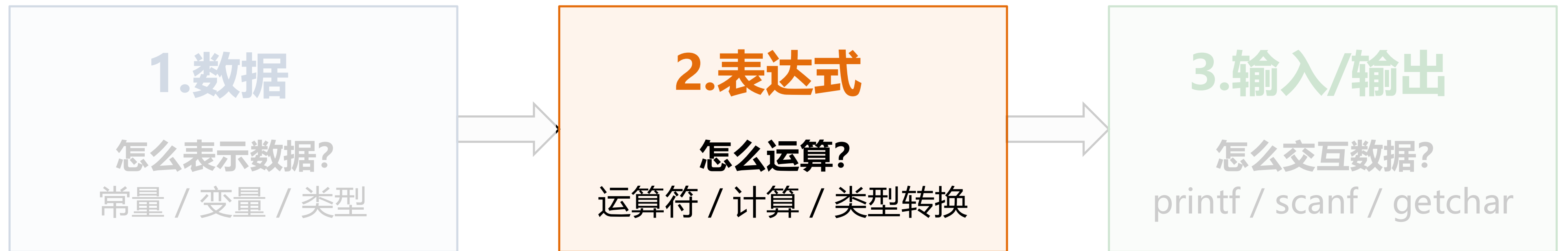
网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2026年秋季

本节内容

□ 如何写出能处理数据的程序？



求 $ax^2 + bx + c = 0$ 的根

```
#include <stdio.h>
#include <math.h>

int main(void){
    float a, b, c;
    float disc, x1, x2, p, q;

    scanf("%f%f%f", &a, &b, &c);

    disc = b * b - 4 * a * c;
    p = -b / (2.0 * a);
    q = sqrt(disc) / (2.0 * a);
    x1 = p + q;
    x2 = p - q;

    printf("x1=%7.2f\nx2=%7.2f\n", x1, x2);
}
```

数据

表达式

输入输出

求 $ax^2 + bx + c = 0$ 的根

```
#include <stdio.h>
#include <math.h>

int main(void){
    float a, b, c;
    float disc, x1, x2, p, q;

    scanf("%f%f%f", &a, &b, &c);

    disc = b * b - 4 * a * c;
    p = -b / (2.0 * a);
    q = sqrt(disc) / (2.0 * a);
    x1 = p + q;
    x2 = p - q;

    printf("x1=%7.2f\nx2=%7.2f\n", x1, x2);
}
```

- 常量
- 变量
- 子表达式
- 函数调用
-

表达式： 运算符和操作数的序列，
具有值和类型

从运算到运算符

- 运算在数学上是一种行为，是通过已知量的可能的组合获得新的量
- 运算的本质是集合之间的映射，**运算符就是反映运算类型的符号**
 - 例如，算术中的加法 $5 + 3 = 8$ ，这里 5 和 3 是操作数，8 是结果，而加号 “+” 是运算符表明这是一个加法运算
 - 这是一个常见的二元运算，本质上是 $a \times b \rightarrow C$ 形式的映射
- C语言内置了丰富的运算符，如**算术运算符、赋值运算符、逻辑运算符、关系运算符、位运算符**等等

常用运算符

1	算术运算符	+ - * / % ++ --
2	关系运算符	> < == >= <= !=
3	逻辑运算符	! &&
4	位运算符	<< >> ~ ^ &
5	赋值运算符	=及其扩展赋值运算符
6	条件运算符	? :
7	逗号运算符	,
8	指针运算符	* &
9	求字节数运算符	sizeof
10	强制类型转换运算符	(类型)
11	成员运算符	. ->
12	下标运算符	[]
13	其他	如函数调用运算符()

本节重点

类别	运算符
算术运算	+ - * / %
赋值运算	=
复合赋值	+= -= *= /= %=
自增、自减	++ --
类型转换	(类型)

常用算数运算符

运算符	含义	举例	结果
+	正号运算符 (单目运算符)	$+ a$	a的值
-	负号运算符 (单目运算符)	$-a$	a的算术负值
*	乘法运算符	$a * b$	a和b的乘积
/	除法运算符	a / b	a除以b的商
%	求余运算符	$a \% b$	a除以b的余数
+	加法运算符	$a + b$	a和b的和
-	减法运算符	$a - b$	a和b的差

练习

□ 已定义 `double x, y;` 请把下面数学公式写成表达式

$$y = \frac{x + 3}{2}$$

$$y = x^2 + 2x + 1$$

$$y = \frac{3x + 5}{x - 2}$$

$$y = (x + 3) / 2;$$

$$y = x * x + 2 * x + 1;$$

$$y = (3 * x + 5) / (x - 2);$$

求商与取余

- “/” 分为**整数除法**和**浮点除法**，由操作数类型决定
 - 若操作数是整数，执行整数除法，结果是整数，**小数部分被截去**
 - 若操作数是浮点数，执行浮点除法，结果是浮点数

```
printf("%d\n", 5/2);
```

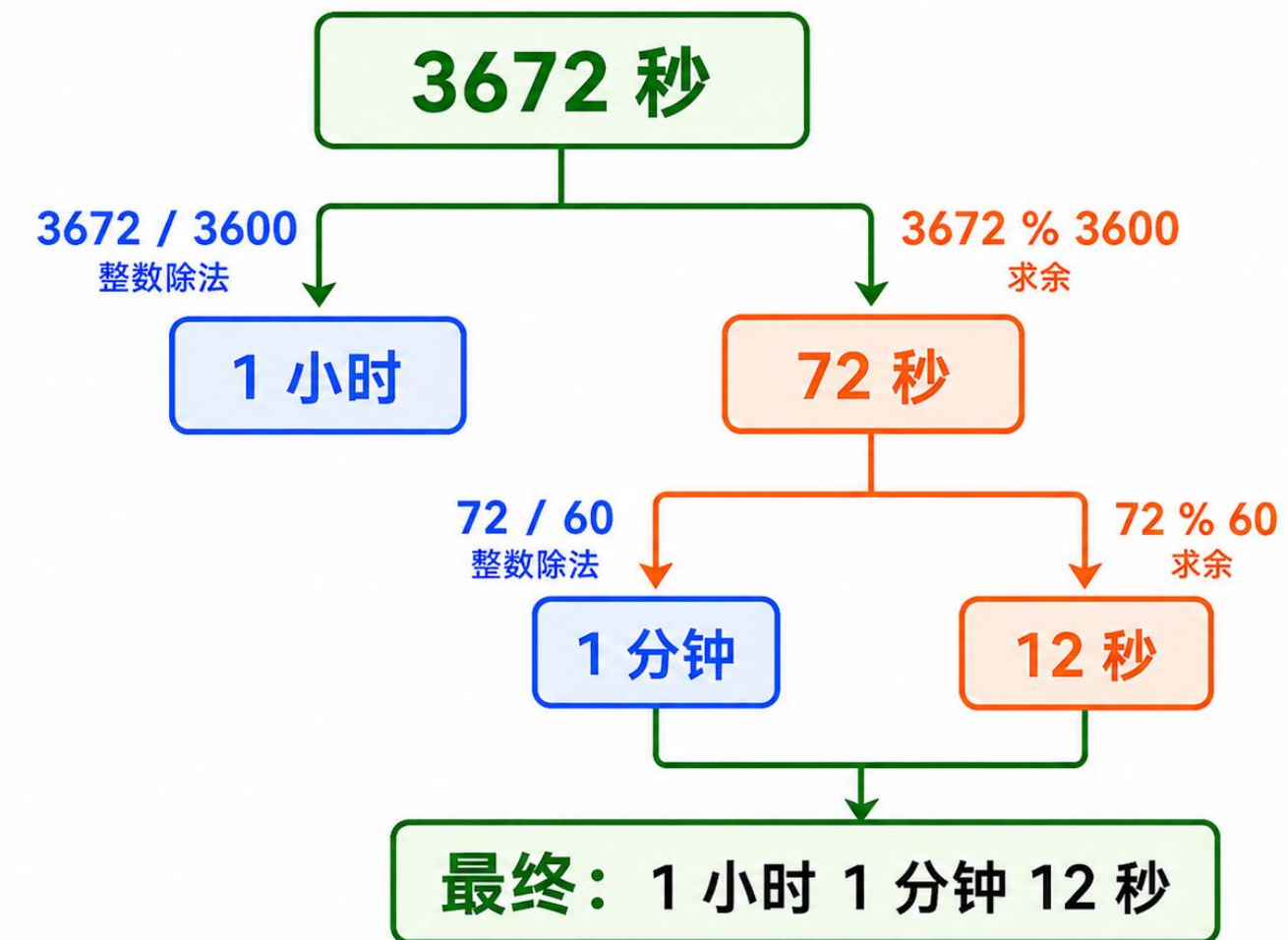
```
printf("%f\n", 5.0/2);
```

- “%” 要求**操作数为整数**，结果也是整数
 - $a\%b$ 的值等于 $a-(a/b)*b$

整数除法和取余

□ 3672 秒是多少小时、分钟和秒？

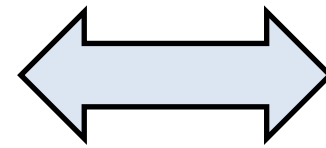
- 小时 = 总秒数 / 3600
- 剩余秒数 = 总秒数 % 3600
- 分钟 = 剩余秒数 / 60
- 秒 = 剩余秒数 % 60



运算符的优先级与结合性

□ **优先级**：一个表达式里有多多个运算符，谁优先级更高？

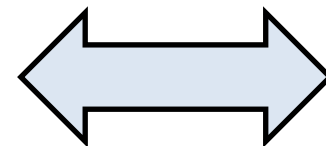
```
int x = 7 + 3 * 2;
```



```
int x = 7 + (3 * 2);
```

□ **结合性**：多个优先级相同的运算符共享同一操作数，先执行哪个？

```
16 / 4 * 2
```



```
(16 / 4) * 2
```

□ 从左向右组合的叫**左结合性**，从右向左的叫**右结合性**

■ `"/"` 和 `"*"` 都是左结合性，所以 `16/4*2` 等价于 `(16/4)*2`

运算符的优先级与结合性

类别	运算符	结合性
后缀	() [] -> . ++ --	从左到右
一元	+ - ! ~ ++ -- (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 aND	&	从左到右
位异或 XOR	^	从左到右
位或 OR		从左到右
逻辑与 aND	&&	从左到右
逻辑或 OR		从左到右
条件	?:	从右到左
赋值	= += -= *= /= %= >>= <<= &= ^= =	从右到左
逗号	,	从左到右

表达式的类型与自动类型转换

□ 为什么同一个 “/” 会得到不同结果？

```
printf("%d\n", 5/2);
```

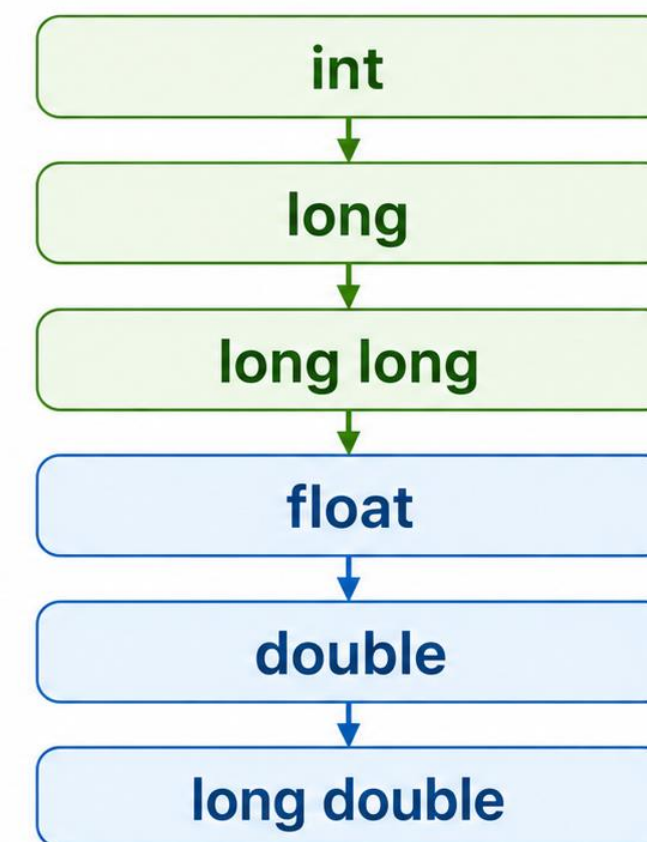
```
printf("%f\n", 5.0/2);
```

□ 当操作数类型不同，C会根据规则进行**隐式类型转换**，再完成运算

类型转换

□ 隐式类型转换：编译器自动决定的类型转换

1. 在赋值表达式中，计算结果会转换为被赋值变量类型
2. 两种类型运算时，两个值会转换为两种类型的更高级别



□ 强制类型转换：程序员主动指定的类型转换，将一个值转换成指定数据类型

语法：(类型名)表达式

```
int a = 21, b = 4;  
double c, d;
```

```
c = (double) a / b;  
d = (double)(a / b);
```

```
printf("c = %f\n", c);  
printf("d = %f\n", d);
```

赋值运算与赋值表达式

□ “=” 称为赋值运算符，表示把右侧表达式的值存入左侧变量，必要时进行类型转换

□ 赋值运算符**左侧**须是一个**可被修改的存储位置**

int a = 5.2;



5 = a;



a + b = 5;



□ **赋值表达式的值**等于赋值后左侧变量的值

赋值运算与赋值表达式

□ a、b、c的值如何变化？最后输出多少？

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int a = 1, b = 1, c = 1;
```

```
    a = b = c = 5;
```

```
    a = 5 + (c = 6);
```

```
    a = (b = 4) + (c = 6);
```

```
    a = (b = 10) / (c = 2);
```

```
    a = (b = 3 * 4);
```

```
    printf("%d\n", a = b = c = 1);
```

```
}
```

→ 表达式值为5

→ 表达式值为11

→ 表达式值为10

→ 表达式值为5

→ 表达式值为12

→ 表达式值为?

a b c均为5

a为11, c为6

a为10, b为4, c为6

a为5, b为10, c为2

a b值均为12

a b c 均为1

复合赋值运算符

- 在赋值运算符之前加上其他运算符，可以构成复合的运算符

完整写法	复合赋值
$a = a + b$	$a += b$
$a = a - b$	$a -= b$
$a = a * b$	$a *= b$
$a = a / b$	$a /= b$
$a = a \% b$	$a \% = b$

- 二元运算符，都可以与赋值符一起组合成复合赋值符
- 注意：如果赋值符右边是包含**若干项的表达式**，则相当于它有**括号**
 - $x \% = y + 3$ 等价于 $x = x \% (y + 3)$

自增与自减

□ 程序中 $i = i + 1$ 这类自增/减 1 的表达式很常见，C 提供更简洁写法

■ $++i$, $--i$: 在**使用 i 之前**，先使 i 的值加/减 1

■ $i++$, $i--$: 在**使用 i 之后**，再使 i 的值加/减 1

```
int i=3,j;  
j=++i;//i的值先变成4,再赋给j,j的值为4
```

```
int i=3,j;  
j=i++;//先将i的值3赋给j,j的值为3,然后i变为4
```

```
int i=10;  
printf("%d",++i);
```

```
int i=10;  
printf("%d",i++);
```

□ 谨慎使用 $++$ 和 $--$ 运算符，只用最简单的 $i++$ 、 $i--$ ，且把它们作为单独的表达式

思考

□ 以下表达式的值?

1. `int i=1; printf("%d %d\n", i*i++, i);`
2. `j = i++ + i++;`

□ 行为取决于编译器

练习

□ 给定整数 a、b，如何交换它们的值？

```
#include <stdio.h>

int main(void) {
    int a, b;
    scanf("%d%d", &a, &b);

    // ?

    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```

有没有其他办法？

交换两个数

```
#include <stdio.h>

int main(void) {
    double a, b;
    scanf("%lf%lf", &a, &b);

    a = a + b;
    b = a - b;
    a = a - b;

    printf("a=%lf, b=%lf\n", a, b);

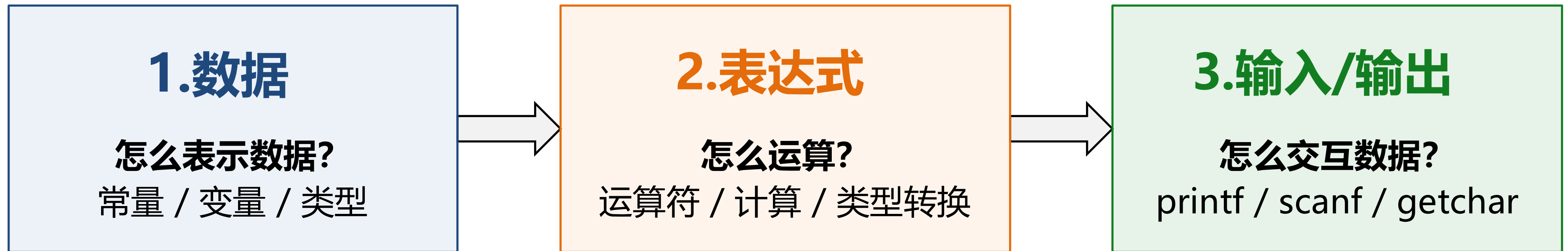
    return 0;
}
```

变量	a	b
初始值	x	y
a=a+b	x+y	y
b=a-b	x+y	x
a=a-b	y	x

这么做可能有什么问题？

总结

□ 如何写出能处理数据的程序？



作业

- 阅读《C程序设计》第3章
- 完成 PTA 本周习题

补充

位运算符

□ 位运算符作用于**整数的二进制位**，逐位执行操作

■ 与：&

■ 或：|

■ 异或：^

■ 取反：~

p	q	p & q	p q	p ^ q	~p
0	0	0	0	0	1
0	1	0	1	1	1
1	1	1	1	0	0
1	0	0	1	1	0

位运算符

□ 假设 $a = 60$ (00111100), $b = 13$ (00001101), 则

```
a    = 0011 1100
b    = 0000 1101
-----
a & b = 0000 1100
a | b = 0011 1101
a ^ b = 0011 0001
~ a  = 1100 0011
```

□ 位运算是对运算数的**每一个二进制位**分别进行

位运算符

□ 左移运算符 $\ll$: 将一个整数的二进制位整体左移若干位

■ 左侧移出的位丢弃, 右侧补 0

$a = 0011\ 1100$

$a \ll 2 = 1111\ 0000$

□ 右移运算符 $\gg$: 将一个整数的二进制位整体右移若干位

■ 右侧移出的位丢弃, 左侧补 0 (正数)

$a = 0011\ 1100$

$a \gg 2 = 0000\ 1111$

位运算应用：掩码 (Mask)

- 用一个二进制数作为“掩码”，配合位运算，只操作指定的二进制位，而不影响其他位

A = 1010 1010

B = 0000 1111 掩码

A & B = 0000 1010 清零指定位

A | B = 1010 1111 置1指定位

A ^ B = 1010 0101 翻转指定位

交换两个数

```
#include <stdio.h>

int main(void) {
    int a, b, c;
    scanf("%d%d", &a, &b);

    a = a ^ b;
    b = b ^ a;
    a = a ^ b;

    printf("a=%d, b=%d\n", a, b);

    return 0;
}
```

变量	a	b
初始值	x	y
a=a^b	x^y	y
b=a^b	x^y	x^(y^y)
a=a^b	y	x

关系运算符

□ 关系运算符用于**比较**两个值之间的关系，表达式结果为整数 **0** 或 **1**：
成立为 1（真），不成立为 0（假）

运算符	含义	a=10, b=20
==	判断两边的值是否 相等	a == b → 0 (假)
!=	判断两边的值是否 不相等	a != b → 1 (真)
>	判断左边的值是否 大于 右边	a > b → 0 (假)
<	判断左边的值是否 小于 右边	a < b → 1 (真)
>=	判断左边的值是否 大于等于 右边	a >= b → 0 (假)
<=	判断左边的值是否 小于等于 右边	a <= b → 1 (真)

逻辑运算符

□ 逻辑运算符用于对一个或多个条件进行**逻辑运算**，运算结果为整数 **0** 或 **1**：结果**成立时为 1（真）**，结果**不成立时为 0（假）**

运算符	描述	a=1, b=0
&&	如果两个操作数都 非零 ，则条件为真	a && b 为假
	如果两个操作数中有 任意一个非零 ，则条件为真	a b 为真
!	用来 逆转 操作数的逻辑状态，如果条件为真则逻辑非运算符使其为假	!(a && b) 为真

谢谢